

Module 17: Transactions

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Outline

- Concurrent and Parallel Execution
- Transaction Concept
- Transaction State
- Concurrent Executions
- Serializability
- Recoverability
- Implementation of Isolation
- Transaction Definition in SQL
- Testing for Serializability.

©Silberschatz, Korth and Sudarshan



Transactions

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- E.g., transaction to transfer $50 from account A to account B:
  1. **read**(*A*)
  2. *A* := *A* – 50
  3. **write**(*A*)
  4. **read**(*B*)
  5. *B* := *B* + 50
  6. **write**(*B*)
- Multiple transactions can run at the same time

©Silberschatz, Korth and Sudarshan



Multiple Transactions

- Two modes of execution
  - Concurrent execution
  - Parallel execution

©Silberschatz, Korth and Sudarshan



Transactions

- Two main issues to deal with transactions
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions

©Silberschatz, Korth and Sudarshan



ACID Properties

To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database, or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
  - That is, for every pair of transactions T_i and T_j, it appears to T_i that either T_j, finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.



Atomicity

Definition: Either **all** the operations in a transaction are executed successfully and reflected in the database, or **none** of the operations take effect. If something goes wrong during the transaction, the system rolls back any changes.

Example: Consider a bank transaction where you transfer $100 from **Account A** to **Account B**.

- Operation 1: Deduct $100 from **Account A**.
- Operation 2: Add $100 to **Account B**.

If the system successfully deducts $100 from **Account A** but fails to add the money to **Account B** due to a system error, **Atomicity** ensures that the whole transaction is rolled back. The $100 remains in **Account A**, and the transaction never partially executes.

©Silberschatz, Korth and Sudarshan



Consistency

- In a **bank database**, there is a rule that no account can have a negative balance. When a customer tries to withdraw money from their account:
- **Operation**: Deduct the withdrawal amount from the account balance.
- **Example**:
  - **Current balance**: $50
  - **Withdrawal request**: $100
- If a customer tries to withdraw **$100** but only has **$50** in their account, the system should **reject the transaction** and ensure **Consistency** by not allowing the balance to go negative. The transaction would be rolled back, and the account balance would remain unchanged at **$50**.
- This ensures that the **consistency rule**—which prevents negative account balances—is maintained, keeping the database in a valid state before and after the transaction attempt.

©Silberschatz, Korth and Sudarshan



Isolation.

Definition: Even though multiple transactions may be executing at the same time, **Isolation** ensures that each transaction operates as if it were running **alone**. The intermediate state of one transaction should not be visible to other transactions.

Example: Suppose two customers, Alice and Bob, are transferring money between accounts at the same time:

John's Transaction: Transfer $100 from **Account A** to **Account B**.
Jerry's Transaction: Transfer $200 from **Account A** to **Account C**.

If John and Jerry's transactions are running concurrently, **Isolation** ensures that John doesn't see Jerry's intermediate results (such as the $200 deduction from **Account A**), and vice versa. For John, it will appear that Jerry's transaction occurred either completely before or completely after his, preventing any confusion or inconsistent states.

©Silberschatz, Korth and Sudarshan



Durability

Definition: Once a transaction is **committed**, its changes are **permanent** in the database, even if there is a system failure (such as a crash or power outage). The data is safely stored in non-volatile memory (like disk storage).

Example: After successfully transferring $100 from **Account A** to **Account B**, the system crashes. When the system restarts, **Durability** ensures that the $100 transfer is still reflected in the database, i.e., **Account A**'s balance is decreased, and **Account B**'s balance is increased.

©Silberschatz, Korth and Sudarshan



ACID Properties Using Bank Example

- **Atomicity requirement**
  - If the transaction fails after step 3 and before step 6, money will be "lost" leading to an inconsistent database state
    - Failure could be due to software or hardware
  - The system should ensure that updates of a partially executed transaction are not reflected in the database
- **Durability requirement**
  - Once the user has been notified that the transaction has completed (i.e., the transfer of the $50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

©Silberschatz, Korth and Sudarshan



ACID Properties Using Bank Example (Cont.)

- **Consistency requirement**
  - In above example: The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
  - Explicitly specified integrity constraints such as primary keys and foreign keys
  - Implicit integrity constraints
    - Example, sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
- A transaction must see a consistent database.
- During transaction execution the database may be temporarily inconsistent.
- When the transaction completes successfully the database must be consistent
  - Erroneous transaction logic can lead to inconsistency

©Silberschatz, Korth and Sudarshan



ACID Properties Using Bank Example (Cont.)

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1	T2
1. **read**(A)	
2. $A := A - 50$	
3. **write**(A)	
	read(A), read(B), print(A+B)
4. **read**(B)	
5. $B := B + 50$	
6. **write**(B)	

- Isolation can be ensured trivially by running transactions **serially**
  - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

©Silberschatz, Korth and Sudarshan



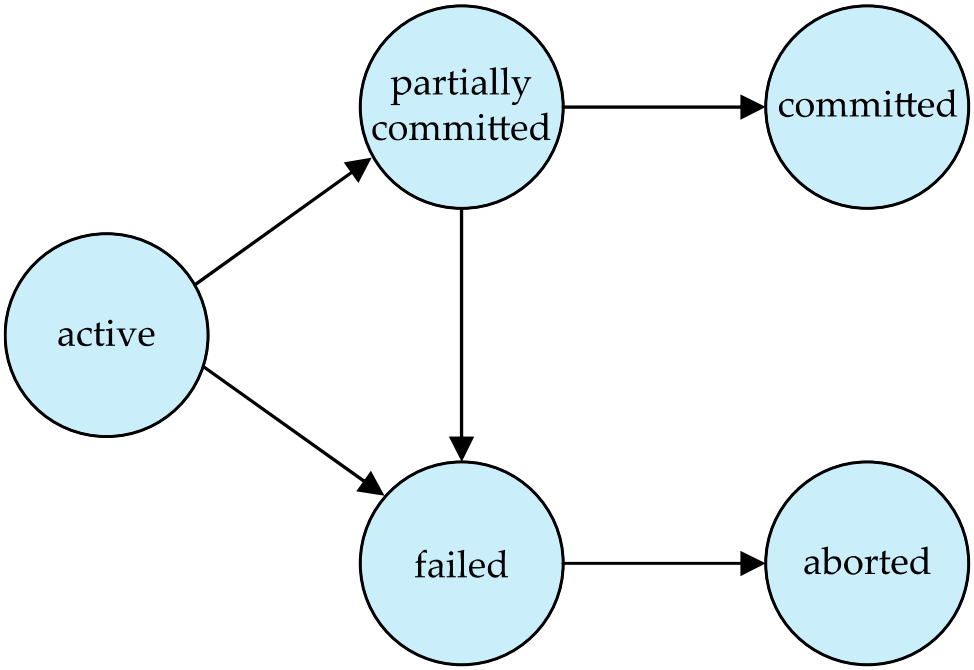
Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
  - Restart the transaction
    - Can be done only if no internal logical error - If the failure was due to a temporary issue, the transaction can be restarted to retry the transfer.
    - For example, if a network issue caused the failure, the transaction can be attempted again.
  - Kill (abort) the transaction

    •If the failure was due to a **logical error** (e.g., insufficient funds in **Account A**), the transaction should be killed permanently.
    •In this case, the transaction will not be restarted because the error is due to a business rule violation.
- **Committed** – after successful completion.



Transaction State (Cont.)





Example of a Complete Transaction:

Scenario: Flight Booking System

- **Atomicity**: A user books a flight and the transaction includes deducting the fare from the user's credit card and confirming the seat. If the seat is confirmed but the payment fails, the seat should not be booked, and the transaction is rolled back.
- **Consistency**: The system checks that the total number of available seats does not exceed the maximum allowed for the flight. The booking transaction ensures that even after the booking, the flight stays within its capacity limit.
- **Isolation**: Two users trying to book the last seat on a flight should not see the intermediate states of each other's transactions. Only one user's transaction should succeed, and the other should fail, keeping the system consistent.
- **Durability**: After the flight is successfully booked, even if the system crashes, the booking details should be saved and persist after the system restarts.

These ACID properties work together to ensure database **reliability**, **data integrity**, and **correctness** in a multi-user, concurrent environment.

©Silberschatz, Korth and Sudarshan



Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
  - **Increased processor and disk utilization**, leading to better transaction *throughput*
    - E.g., one transaction can be using the CPU while another is reading from or writing to the disk
  - **Reduced average response time** for transactions: short transactions need not wait behind long ones.
- Concurrency versus parallelism
  - Concurrency example: A supermarket with a single cashier and several lines of customers ordering from the single cashier (lines take turns ordering).
  - Parallelism example: A supermarket with a multiple cashier and several lines of customers, with each line getting its own cashier.

©Silberschatz, Korth and Sudarshan



Concurrent Control Scheme

- A mechanisms to achieve isolation
- That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database
- Will study in Chapter 18, after studying notion of correctness of concurrent executions.

©Silberschatz, Korth and Sudarshan



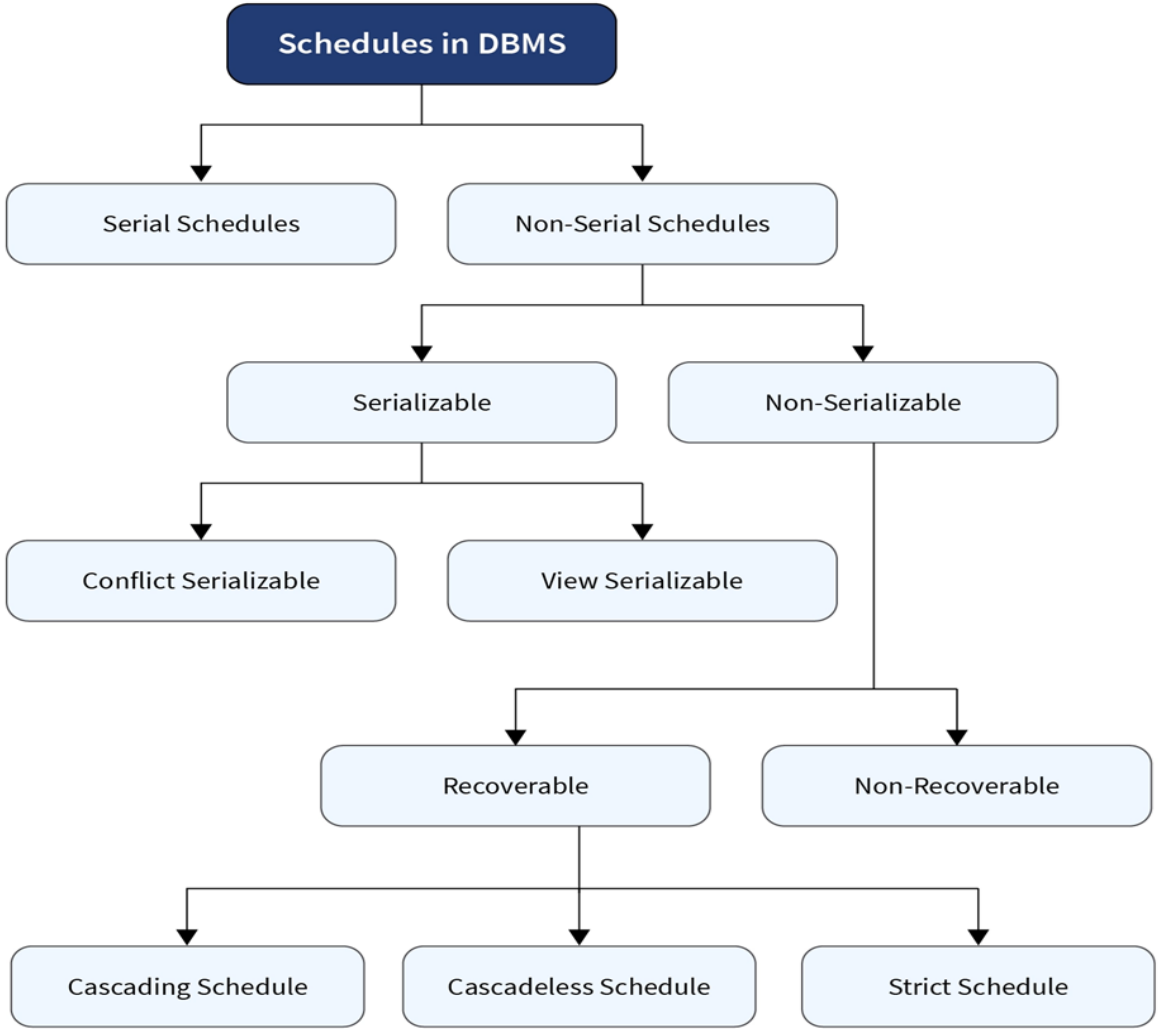
Schedule

- A sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
  - A schedule for a set of transactions must consist of all instructions of those transactions
  - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
  - By default, transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

©Silberschatz, Korth and Sudarshan



Types of Schedules



©Silberschatz, Korth and Sudarshan



Serial Schedule:

Definition: In a **serial schedule**, transactions are executed one after another, without any overlap. In this type of scheduling, one transaction must complete its execution before another transaction starts.

Characteristics:

- Ensures consistency and isolation.
- No interleaving of operations from different transactions.
- Safe from conflicts, as transactions do not overlap.

Advantage:

•Guarantees consistency and correctness because transactions are executed sequentially.

Disadvantage:

•Not efficient in terms of performance, as it doesn't allow concurrent execution of transactions.



Non-Serial Schedule:

Definition: A **non-serial schedule** allows the interleaving of operations from different transactions. Multiple transactions can execute in parallel or overlap in time, but the order of execution may lead to conflicts if not carefully managed.

Characteristics:

- Operations of multiple transactions are interleaved.
- May or may not preserve consistency, depending on how the transactions interact.

Advantage:

•More efficient, as it allows concurrent execution, leading to better resource utilization.

Disadvantage:

•Can lead to **conflicts** (like dirty reads, lost updates, etc.) if proper concurrency control mechanisms (like locking or timestamp ordering) are not applied.

•May result in inconsistent database states if conflicts are not properly resolved.



Serial Schedule	Non-Serial Schedule
Transactions are executed one after another (no overlap).	Transactions are interleaved or overlap.
Always ensures consistency and isolation.	May lead to conflicts without concurrency control.
Slower execution as transactions are sequential.	Faster and more efficient due to parallelism.
Simple and easy to understand.	Complex, requires concurrency control techniques.
No concurrency, thus safe from anomalies.	Can lead to anomalies (like dirty reads or lost updates).



Schedule 1

- Let T_1 transfer \$50 from *A* to *B*, and T_2 transfer 10% of the balance from *A* to *B.*
- A serial schedule in which T_1 is followed by T_2 :

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit



Schedule 2

- A serial schedule where T_2 is followed by T_1

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	



Are the Results of Both Schedules the Same

- Consistency requirement:
  - The sum “A+B” should be the same at the end of both executions
- Assume initially A = 1,000, B = 200
  - Schedule 1. T1 A = 950. B = 250

    T2 A = 855. B = 345 A+B =1,200
  - Schedule 2. T2 A = 1,100 B = 100

    T1 A = 1,050 B = 150 A+B =1,200
- But each transaction sees different values of A and B
  - Are they both correct?
  - Which one is “better”
- Another example --Two transaction trying to grab the last ticket to a concert.

©Silberschatz, Korth and Sudarshan



Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following **schedule is not a serial schedule**, but it is *equivalent* to Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

- In Schedules 1, 2 and 3, the sum A + B is preserved.



Schedule 4

- The following concurrent schedule does not preserve the sum of ($A + B$).

T_1	T_2
read (A)	
$A := A - 50$	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	$B := B + temp$
	write (B)
	commit



Serial Schedule

(a)

T1	T2

read(A);
A := A – N;
write(A);
read(B);
B := B + N;
write(B);	

	read(A);
	A := A + M;
	write(A);

Time

Schedule A

(b)

T1	T2

	read(A);
	A := A + M;
	write(A);

read(A);
A:= A - N;
write(A);
read(B);
B:= B +N;
write(B);	

Time

Schedule B

Non - Serial Schedule

(c)

T1	T2
read(A); A := A – N;	
	read(A); A := A + M;
write(A); read(B);	
	write(A);
B := B + N; write(B);	

Time

Schedule C

(d)

T1	T2

read(A);
A := A – N;
write(A);	

	read(A);
	A := A + M;
	write(A);

read(B);
B := B + N;
write(B);	

Time

Schedule D



Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:

1. **Conflict serializability**

   If a non-serial schedule can be transformed into a serial schedule by swapping non-conflicting operations, then the schedule is considered **conflict serializable**.

2. **View serializability**



Simplified view of transactions

- We ignore operations other than **read** and **write** instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.

©Silberschatz, Korth and Sudarshan



Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions wrote Q.
  1. I_i = **read**(Q), I_j = **read**(Q). I_i and I_j don't conflict.
  2. I_i = **read**(Q), I_j = **write**(Q). They conflict.
  3. I_i = **write**(Q), I_j = **read**(Q). They conflict
  4. I_i = **write**(Q), I_j = **write**(Q). They conflict
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

©Silberschatz, Korth and Sudarshan



Conflicts

- Read(A)- **Write (A)**
- **Write (A)–** Read (A)
- Write(A)- **Write (A)**

Not Conflicts

- Read(A)- Read (A)
- Read(A)- Read (B)
- Read(A)- Write (B)
- Write (A)– Read (B)
- Write(A)- Write (B)

©Silberschatz, Korth and Sudarshan



Non Serial to Serial schedule by swapping

- **Method To Swap Non-Conflict Pairs**
- Find the Adjacent Non-conflict pairs.
- Swap the non-conflict pairs.
- **Condition for swapping**
- Just swap position of non-conflict pairs in the transaction

The concept of **non-conflicting pairs** is essential to concurrency control because it allows multiple transactions to execute simultaneously while still maintaining data consistency.

Conditions for Non-Conflicting Operations:

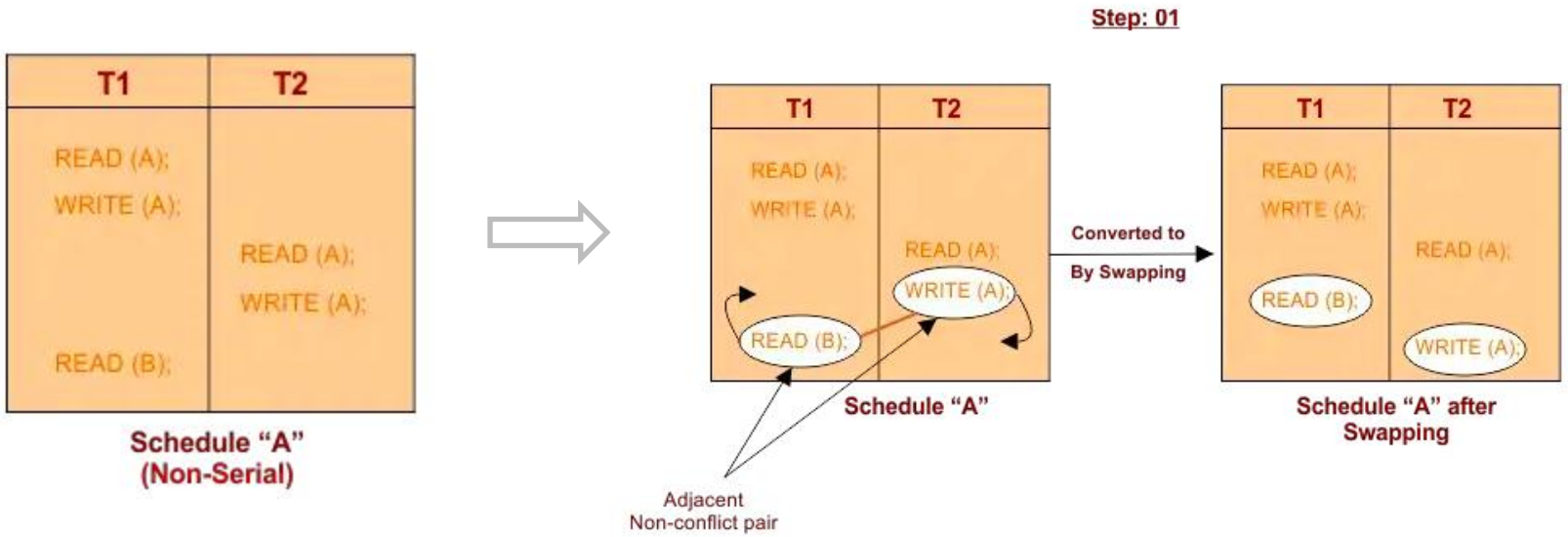
Two operations from different transactions are **non-conflicting** if:
1.They operate on **different data items**.
2.At least one of the operations is a **read** operation.

Steps to Swap Non-Conflict Pairs:

1.Identify adjacent non-conflicting operations from two different transactions.
2.Swap their positions if they meet the non-conflict condition, meaning they don't interfere with each other.

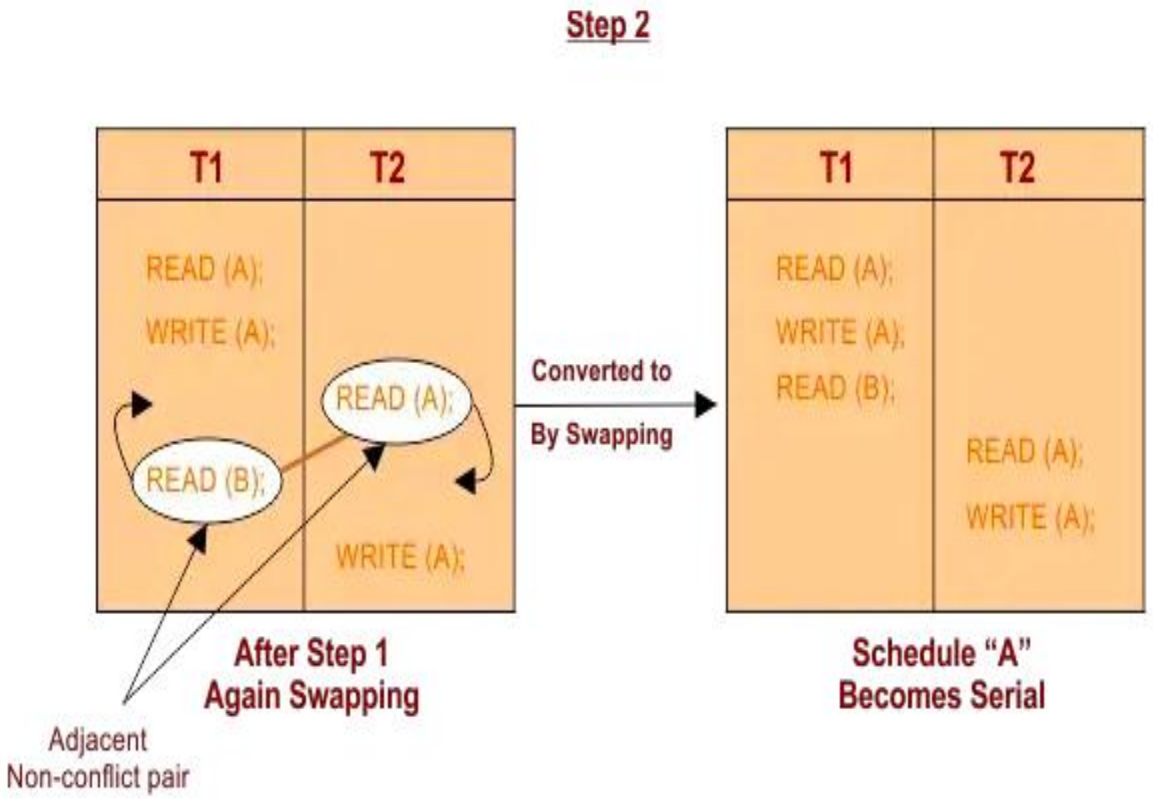
©Silberschatz, Korth and Sudarshan

Example



 ©Silberschatz, Korth and Sudarshan





This is a serial schedule



Conflict Serializability

- If a schedule *S* can be transformed into a schedule *S'* by a series of swaps of non-conflicting instructions, we say that *S* and *S'* are **conflict equivalent**.
- We say that a schedule *S* is **conflict serializable** if it is conflict equivalent to a serial schedule.



Conflict Serializability Example

- Schedule 3 (slide 17,16) can be transformed into Schedule 6, which is a serial schedule where T_2 follows T_1, by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

©Silberschatz, Korth and Sudarshan



Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule < T_3, T_4 >, or the serial schedule < T_4, T_3 >.



Example 2

(There is no
Adjacent Non-conflict pair
exist in Schedule "X")

T1	T2
READ (A);	
WRITE (A);	
	READ (A);
WRITE (A);	
READ (B);	

Schedule "X"

Cannot convert to serial schedule

©Silberschatz, Korth and Sudarshan



View Serializability

- View serializability is that a non-serial schedule produces the same results as a serial schedule, even if the operations are interleaved.

View Serializable vs. Conflict Serializable:

- **Conflict serializability** is stricter and focuses on whether non-conflicting operations can be swapped to achieve a serial schedule.
- **View serializability** is more flexible and allows for a schedule to be serializable as long as the transactions produce the same reads and writes as a serial schedule, even if the operations cannot be swapped.

Note: Every **conflict serializable** schedule is also **view serializable**, but not every **view serializable** schedule is **conflict serializable**.

©Silberschatz, Korth and Sudarshan



View Serializability

- Let *S* and *S'* be two schedules with the same set of transactions. *S* and *S'* are **view equivalent** if the following three conditions are met, for each data item *Q,*

  1. If in schedule S, transaction T_i reads the initial value of *Q*, then in schedule *S'* also transaction T_i must read the initial value of *Q.*
  2. If in schedule S transaction T_i executes **read**(*Q*), and that value was produced by transaction T_j (if any), then in schedule *S'* also transaction T_i must read the value of *Q* that was produced by the same **write**(Q) operation of transaction T_j .
  3. The transaction (if any) that performs the final **write**(*Q*) operation in schedule *S* must also perform the final **write**(*Q*) operation in schedule *S'.*

  Two schedules are view equivalent if they have the same initial reads, final writes, and intermediate reads of written values.

- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.



View Serializability (Cont.)

- A schedule *S* is **view serializable** if it is view-equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes.**



Blind writes

- **Blind writes** refer to a scenario in database transactions where a transaction writes to a data item without first reading its current value. In other words, a "blind write" happens when the transaction updates or overwrites the value of a data item without checking its current state beforehand.

Implications of Blind Writes:

- **Lack of Validation:** The transaction doesn’t check the existing state of the data item, which can lead to overwriting critical information without considering its previous state.
- **Potential Data Anomalies:** Blind writes can result in data inconsistency if they are not properly managed within concurrent transactions. For instance, one transaction might overwrite changes made by another transaction without knowing the prior value.
- **Usefulness:** In some cases, blind writes are deliberate and useful when a transaction does not care about the current state of the data item (e.g., resetting a counter or writing a default value).

Blind writes are common in scenarios where the goal is to overwrite or reset the value of a data item without considering its current value.



Other Notions of Serializability

- The schedule below produces same outcome as the serial schedule $< T_1, T_5 >$, yet is not conflict equivalent or view equivalent to it.

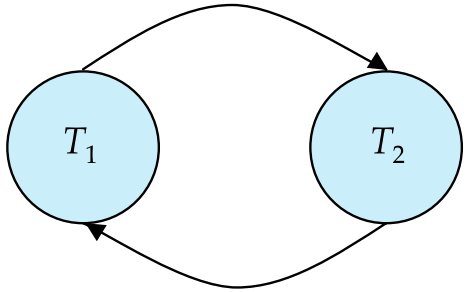
T_1	T_5
read (A)	
$A := A - 50$	
write (A)	
	read (B)
	$B := B - 10$
	write (B)
read (B)	
$B := B + 50$	
write (B)	
	read (A)
	$A := A + 10$
	write (A)

- Determining such equivalence requires analysis of operations other than read and write.



Testing for Conflict Serializability

- Consider some schedule of a set of transactions T_1, T_2, ..., T_n
- **Precedence graph** — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transactions conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example of a precedence graph

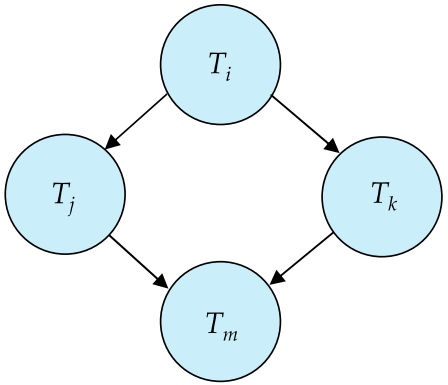


 ©Silberschatz, Korth and Sudarshan

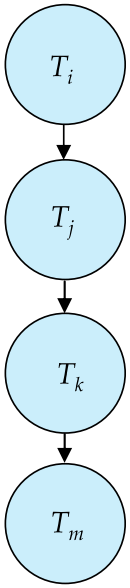


Testing for Conflict Serializability (Cont.)

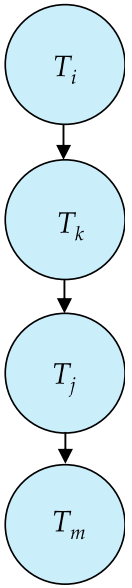
- A schedule is conflict serializable if and only if its precedence graph is acyclic.
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.
  - (Better algorithms take order $n + e$ where e is the number of edges.)
- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
  - This is a linear order consistent with the partial order of the graph.
  - For example, a serializability order for Schedule A would be Schedules B and C.
    - Are there others?



(a)



(b)



(c)

©Silberschatz, Korth and Sudarshan



Test for View Serializability

- The precedence graph test for conflict serializability cannot be used directly to test for view serializability.
  - Extension to test for view serializability has cost exponential in the size of the precedence graph.
- The problem of checking if a schedule is view serializable falls in the class of *NP*-complete problems.
  - Thus, existence of an efficient algorithm is *extremely* unlikely.
- However, practical algorithms that just check some **sufficient conditions** for view serializability can still be used.

©Silberschatz, Korth and Sudarshan



Recoverable Schedules

- Need to address the effect of transaction failures on concurrently running transactions
  - **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i, then the commit operation of T_i appears before the commit operation of T_j.
  - The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

  - If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.



Recoverable Schedule

- A schedule is recoverable if each transaction commits only after all the transactions from which it has read have committed.
- In other words, if some transaction Ty reads a value that has been updated/written by some other transaction Tx, then the commit of Ty must occur after the commit of Tx.

©Silberschatz, Korth and Sudarshan



Recoverable Schedule

- A transaction performs **a dirty read operation from an uncommitted transaction** and **its commit operation is delayed till the uncommitted transaction either commits or roll back** then such a schedule is called as a **Recoverable Schedule**.

- If in a schedule**, failure of one transaction causes several other dependent transactions to rollback or abort**, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**.

©Silberschatz, Korth and Sudarshan



IRRECOVERABLE

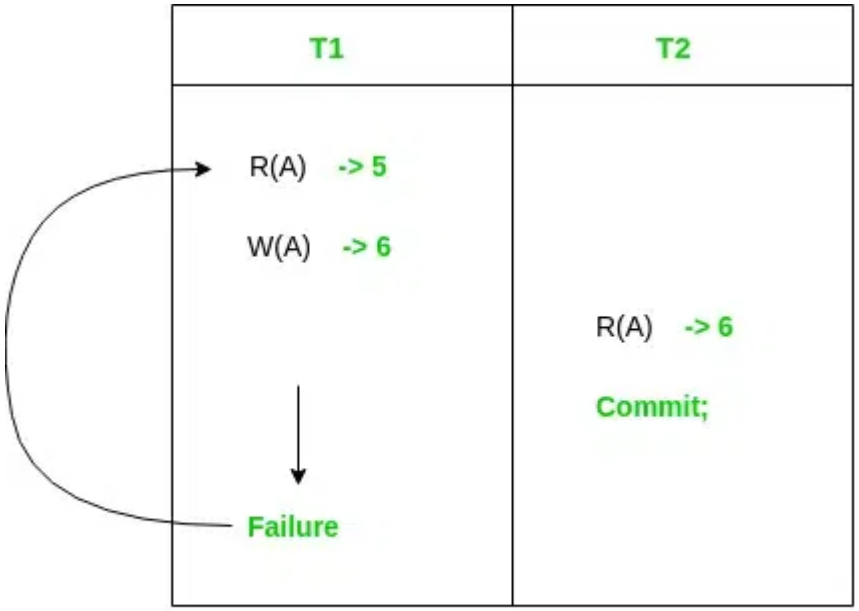


Figure - Dirty Read Problem

A committed transaction should never be rollback. It means that reading value from uncommitted transaction and commit it will enter the current transaction into inconsistent or unrecoverable state this is called Dirty Read problem.



IRRecoverable Schedule

T1	T2
Read(X)	
X = + 20	
Write (X)	
	Read(X)
	X = X + 100
	Write(X)
	Commit
Failed!	
Rollback	

T1 – X is read
X is updated

T2 – X is Read
X is updated
Commit

Irrecoverable schedule

This schedule is irrecoverable because it cannot be successfully rolled back even after the failure of one of the transaction.



Recoverable Schedule

T1	T2
----	----
Read(X)	
X = + 20	
Write (X)	
	Read(X)
	X = X + 100
	Write(X)
Commit	
	Commit

T1 – X is read
X is updated

T2 – X is Read
X is updated

T1 - Commit
T2 – Commit



Recoverable Schedule

Case 1:



In case 1

T1 is failed T2 can be roll backed and T1

Case 2:



In case 2

T2 is failed T2 can be roll backed.

©Silberschatz, Korth and Sudarshan



Cascading Rollbacks

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

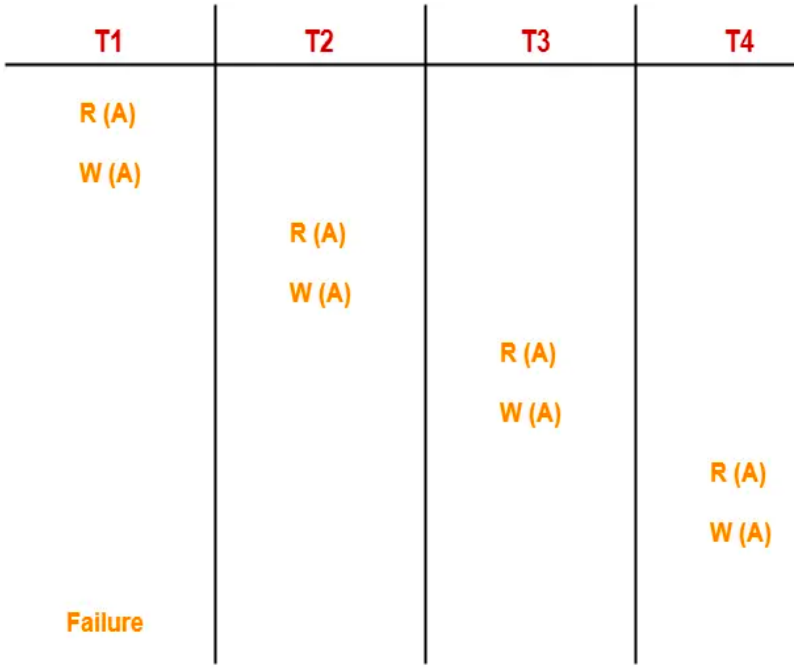
T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A)		
	read (A)	
	write (A)	
		read (A)
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work



Cascading Rollback



Cascading Recoverable Schedule

- Transaction T2 depends on transaction T1.
- Transaction T3 depends on transaction T2.
- Transaction T4 depends on transaction T3.

In this schedule,

- The failure of transaction T1 causes the transaction T2 to rollback.
- The rollback of transaction T2 causes the transaction T3 to rollback.
- The rollback of transaction T3 causes the transaction T4 to rollback.

Such a rollback is called as a **Cascading Rollback**.



Cascadeless Schedules

- **Cascadeless schedules** — cascading rollbacks cannot occur;
  - For each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the read operation of T_j.
- Every cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless



Cascadeless Schedule

T1	T2	T3
R (A)		
W (A)		
Commit		
	R (A)	
	W (A)	
	Commit	
		R (A)
		W (A)
		Commit

Cascadeless Schedule

T1	T2
R (A)	
W (A)	
	W (A) // Uncommitted Write
Commit	

Cascadeless Schedule

- Cascadeless schedule allows only committed read operations.
- However, it allows uncommitted write operations.



Strict Schedule

T1	T2
W (A)	
Commit / Rollback	
	R (A) / W (A)

Strict Schedule



Transaction Definition in SQL

- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
  - **Commit work** commits current transaction and begins a new one.
  - **Rollback work** causes current transaction to abort.
- In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
  - Implicit commit can be turned off by a database directive
    - E.g., in JDBC -- connection.setAutoCommit(false);
- Isolation level can be set at database level
- Isolation level can be changed at start of transaction
    - E.g., In SQL **set transaction isolation level serializable**
    - E.g., in JDBC -- connection.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE)



End of Chapter 17

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Concurrency versus Parallelism

- What is concurrent execution?
- What is parallel execution?

©Silberschatz, Korth and Sudarshan